Multi-Hop Functional Encryption

Pratish Datta[1] **Jiaxin Guan**[2] Alexis Korb[3] Amit Sahai[4]

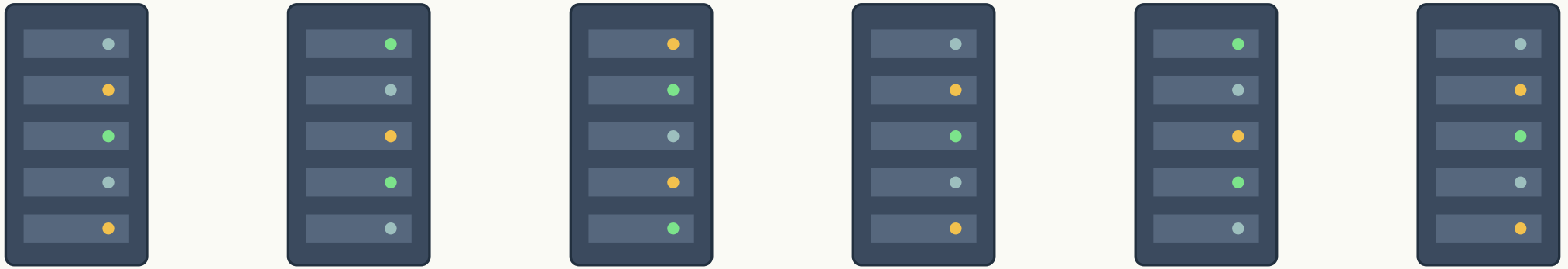
1 NTT Research 2 Zellic 3 Pepperdine University 4 UCLA

TCC 2026 · November 13, 2026

Multi-Hop Functional Encryption

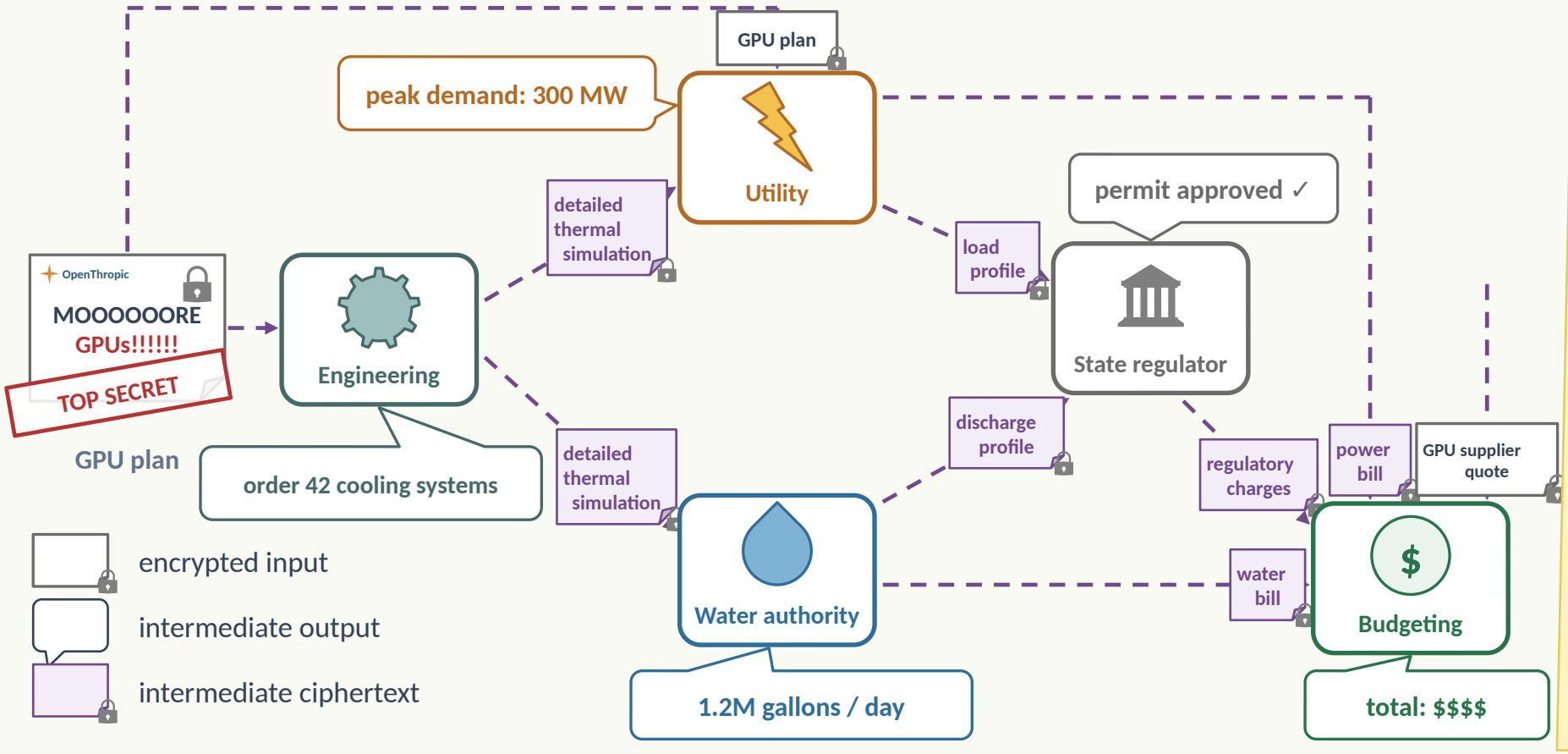
Secure computation over encrypted data in evolving, multi-stage workflows with authorized intermediate outputs.

How to Build an AI Data Center



Disclaimer: OpenThropic is fictional. The authors accept no responsibility for any data center built by following these slides.

How to Build an AI Data Center



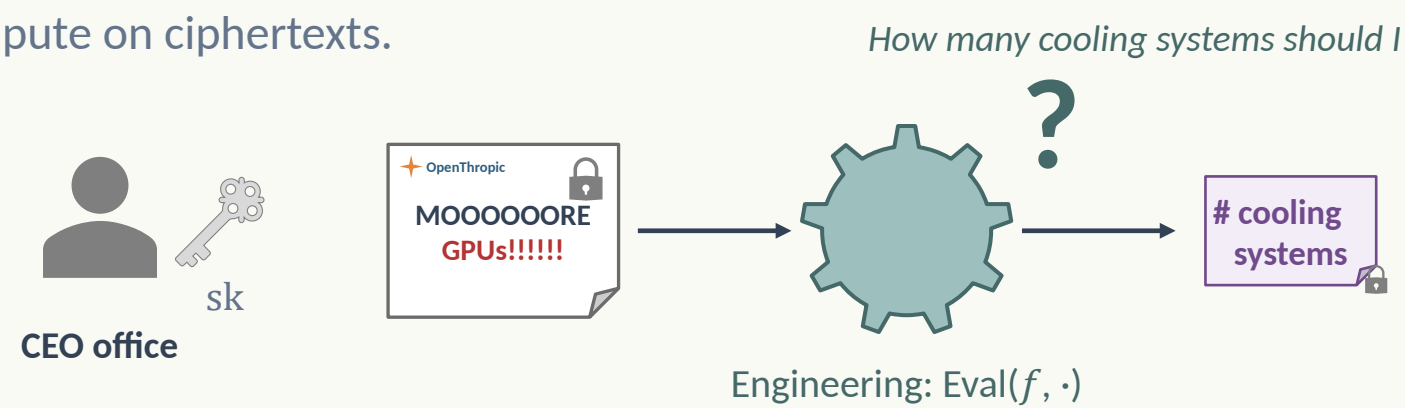
OpenThropic needs:

1. **Intermediate outputs**
   Each department learns a result it can act on immediately.
2. **Need-to-know basis**
   Learns nothing beyond what it needs to act on (not even what it passes on).
3. **Build on earlier work**
   No department redoes upstream work.
4. **Independent operation**
   Each department acts on its own; no coordination needed.
5. **Evolving inputs & workflow**
   New inputs and new steps can be added later.

Idea 1: Fully Homomorphic Encryption

[Gentry 09; Gentry, Halevi, Vaikuntanathan 10]

Anyone can compute on ciphertexts.



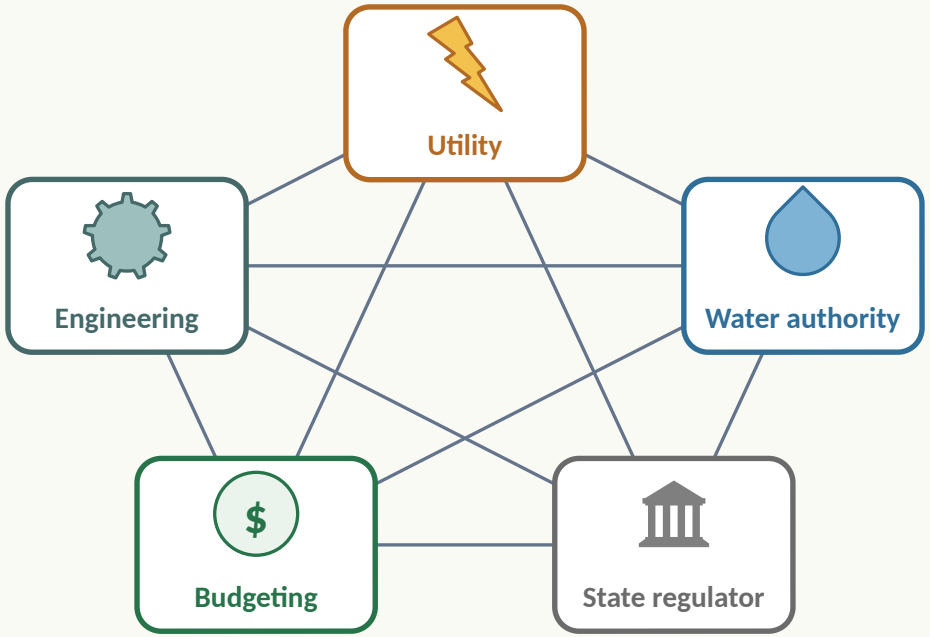
Ask the CEO office to decrypt?

Hand out sk?

Idea 2: Secure Multi-Party Computation

[Yao 86; Goldreich, Micali, Wigderson 87]

Parties compute jointly by talking to each other.



#datacenter-mpc

Utility: Can everyone be online Tuesday, 3 pm?

Water authority: We only meet on the first Monday of the month.

State regulator: Our review starts in six weeks.

Engineering: zzz…

Budgeting: …we haven't been created yet.

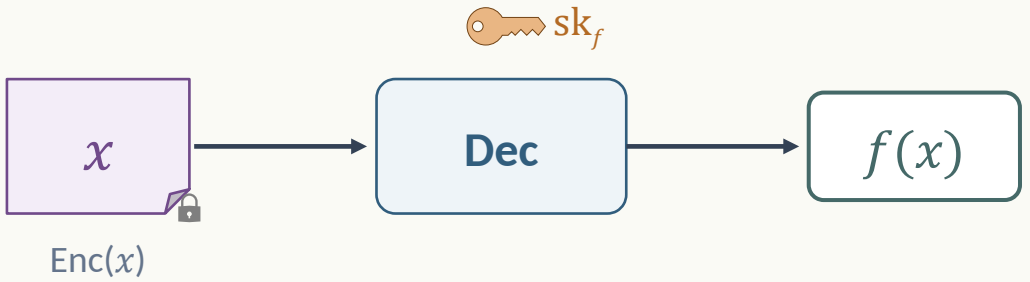
MPC is **interactive**: the departments run a joint protocol and must coordinate with each other.

* Dynamic MPC (e.g. Fluid MPC) supports changing participants and reactive MPC [Canetti, Lindell, Ostrovsky, Sahai 02] supports evolving inputs, but the parties still run a joint protocol together.

Idea 3: Functional Encryption

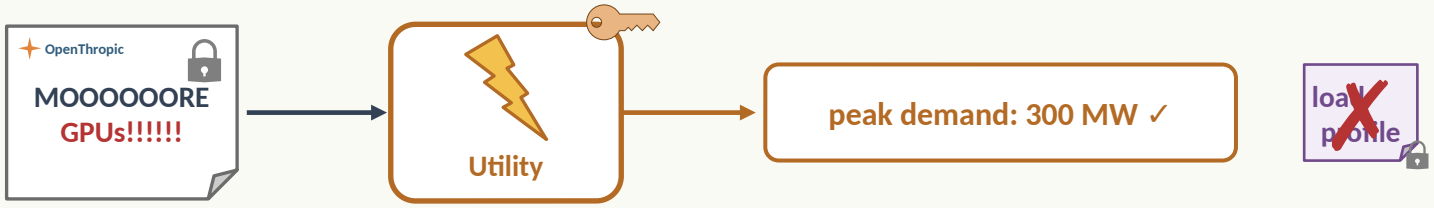
[Boneh, Sahai, Waters 11; O'Neill 10]

Each key is tied to a function; decrypting gives that function evaluated on the data.



The holder of sk_f learns $f(x)$ and nothing else.

Everything that comes out is plaintext.



No intermediate ciphertext to pass on.

Workaround: compose the functions.

Give the regulator a key for $f_{\mathrm{reg}}(f_{\mathrm{util}}(\mathrm{plan}), f_{\mathrm{water}}(\mathrm{plan}))$, and then decrypt directly from the original encrypted input.

Idea 3′: Hard-Code a Public Key?

Fix? Let the function encrypt part of its output under a hard-coded public key.

$f_{\text{util}}(\text{thermal simulation}) = ($ **peak demand** , $\text{Enc}_{\text{pk}}(\text{load profile})$ $)$ **…but whose pk?**

✗ The regulator's PKE key

Too much: the regulator could read the entire load profile, not just decide the permit.

Too early: when the utility's key is issued, the workflow that follows isn't known yet.

✗ FE's own key (chaining FEs)

Needs multi-input FE: FE decrypts a single ciphertext, but the regulator must decrypt the load profile and the discharge profile together.

Needs randomized FE: encrypting takes randomness, so f_{util} is a randomized function.

But: slots must be fixed before the wiring is known; MIFE has only weak simulation security; randomized MIFE [Datta, Guan, Korb, Sahai 25] is secure only when output distributions are extremely close.

This Work

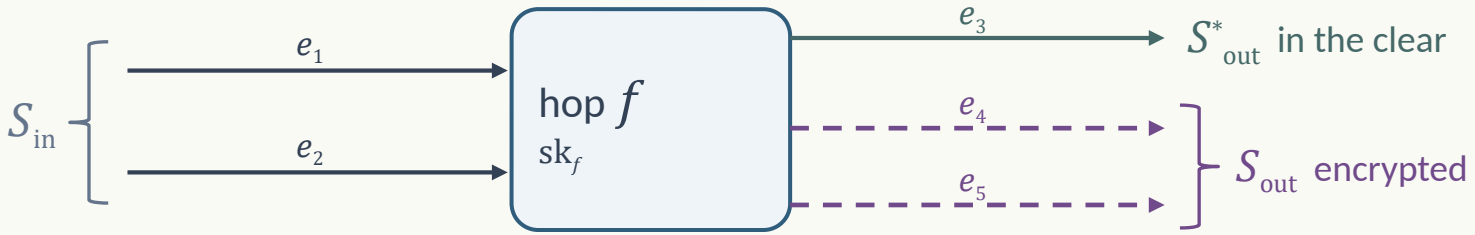
Multi-Hop Functional Encryption (MHFE)

Secure computation over encrypted data in evolving, multi-stage workflows with authorized intermediate outputs.

Our Contributions

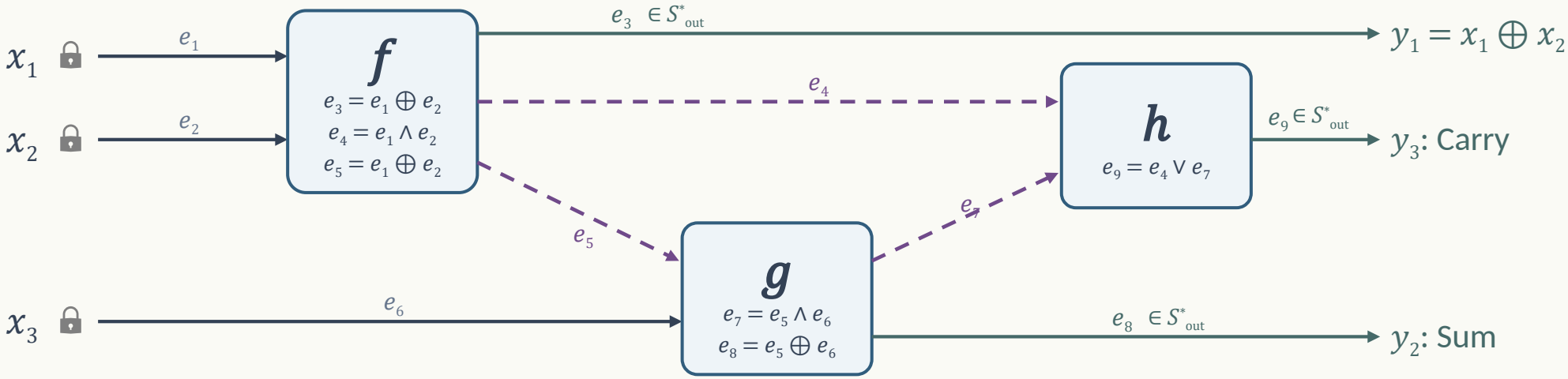
- **Defining Multi-Hop Functional Encryption (MHFE)**

Defining MHFE: Syntax



Setup $\rightarrow$ (mpk, msk)	run once by the key authority
EncSetup(mpk) $\rightarrow$ ek	one encryption key per dataset
Enc(ek, x, e_1) $\rightarrow$ ct_{e1}	encrypt an input onto wire e_1
KeyGen(msk, f, S_{in}, S^*_{out}, S_{out}) $\rightarrow$ sk_f	hop f with its input and output wires
Dec(sk_f, $\{ct_e : e \in S_{in}\}$) $\rightarrow$ $\{y_e : e \in S^*_{out}\}$, $\{ct_e : e \in S_{out}\}$	intermediate outputs; intermediate ciphertexts feed later hops

Example: Adding Three Bits



Step 4

$\mathrm{sk}_h \leftarrow \mathrm{KeyGen}(\mathrm{msk}, h, \{e_4, e_7\}, \{e_9\}, \emptyset)$

$y_3 \leftarrow \mathrm{Dec}(\mathrm{sk}_h, \{\mathrm{ct}_{e4}, \mathrm{ct}_{e7}\}) \quad \Rightarrow \quad y_3 \,\|\, y_2 = x_1 + x_2 + x_3$

Defining MHFE: Security

Simulation (SIM)

The adversary's entire view (ciphertexts, hop keys, forwarded ciphertexts) can be simulated from the authorized clear outputs alone.*

In the example: the adversary's entire view can be simulated from just (y_1, y_2, y_3).

Indistinguishability (IND)

If two datasets give the same authorized clear outputs at every hop, the adversary cannot tell which one was encrypted.*

In the example: $(1, 0, 1)$ vs. $(0, 1, 1)$ both give $(y_1, y_2, y_3) = (1, 0, 1)$, so the adversary cannot distinguish between them.

One-ciphertext SIM security ⇒ many-ciphertext IND security

* A hop key for a node may be requested and used only after the ciphertexts and keys on all of its incoming edges have been fixed in the adversary's view (topological order).

Our Contributions

- Defining Multi-Hop Functional Encryption (MHFE)

- **Building block: Multi-Hop Garbled Circuits (MHGC)**

Multi-Hop Garbled Circuits

Think of it as...

a **single-ciphertext, single-topology, secret-key** MHFE.

...or as

a **garbled circuit with intermediate outputs** that passes garbled wires to the next garbled circuit.

Theorem **One-way functions ⇒ simulation-secure MHGC for P/Poly** (plain model).

Our Contributions

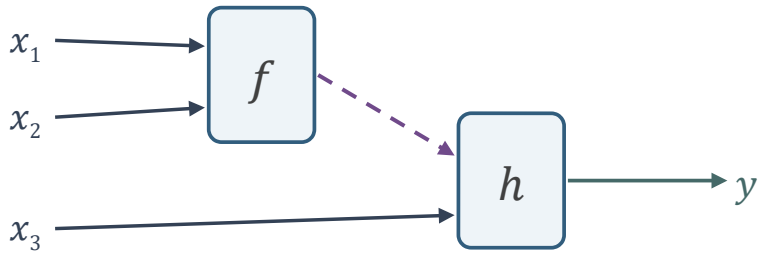
- Defining Multi-Hop Functional Encryption (MHFE)
- Building block: Multi-Hop Garbled Circuits (MHGC)
- **Bootstrapping: Public-key MHFE from any public-key FE**

Bootstrapping to Public-Key MHFE

The challenge: from one topology to many.

MHGC: one topology

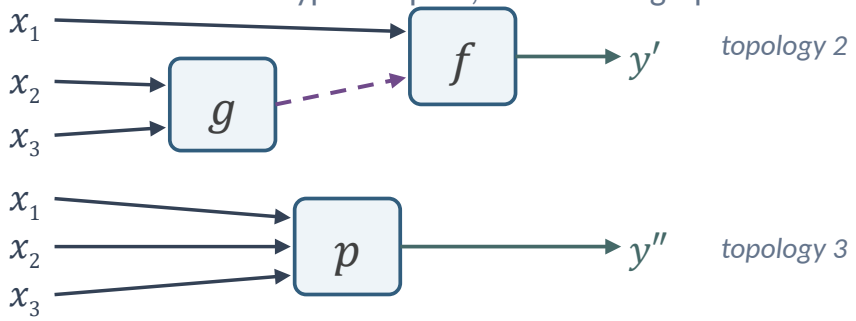
one fixed graph of hops



$\Rightarrow$

MHFE: many topologies

the same encrypted inputs, in different graphs



Idea: for each topology, use public-key FE to generate a fresh MHGC master key.

Theorem Adaptively IND-secure public-key FE for P/Poly + MHGC for P/Poly
$\Rightarrow$ MHFE for P/Poly with **one-ciphertext SIM security** ($\Rightarrow$ many-ciphertext IND security)

Open Problems

1 **Beyond topological order**

Our security asks that each hop key come after the ciphertexts and keys on its incoming wires are fixed. Fully adaptive MHFE?

2 **Removing the central authority**

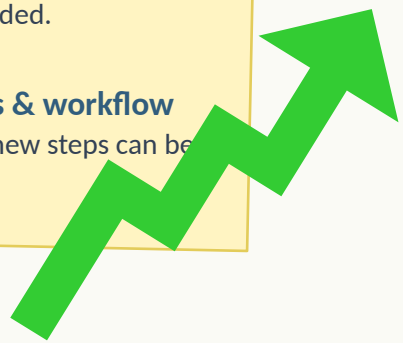
MHFE avoids MPC's interaction at the cost of a key authority. Decentralized or multi-authority MHFE?

3 **Computing across datasets**

Ciphertexts combine only under one encryption key: SIM-secure mixing would imply SIM-secure multi-input FE, which faces strong impossibilities. Weaker notions?

MHFE Provides:

1. **Intermediate outputs**
   Each department learns a result it can act on immediately.
2. **Need-to-know basis**
   Learns nothing beyond what it needs to act on (not even what it passes on).
3. **Build on earlier work**
   No department redoes upstream work.
4. **Independent operation**
   Each department acts on its own; no coordination needed.
5. **Evolving inputs & workflow**
   New inputs and new steps can be added later.



Thank you!

https://eprint.iacr.org/2026/2835